

Zero Trust для ИИ-агентов: почему отдельной идентичности недостаточно

Денис Корбаков · технический директор «Смарт-Софт»

Привет, Хабр! Меня зовут Денис Корбаков, я технический директор «Смарт-Софт». Мы разрабатываем NGFW и регулярно разбираем, какие сетевые события можно использовать для независимого контроля автоматизированных систем. Последний год эта задача резко усложнилась. В инфраструктуре массово появился новый операционный тип машинного субъекта: автономный агент, который сам выбирает инструменты, меняет контекст и делегирует полномочия.

Zero Trust никогда не ограничивался только людьми. [NIST SP 800-207](#) прямо включает в модель non-person entities: сервисы, приложения, автоматизированное ПО, и обсуждает их ещё с версии 2020 года. Субъект как класс не новый, новыми являются масштаб, автономность и модель поведения. Большинство корпоративных реализаций IAM на практике заточены либо под человека с интерактивной сессией, либо под стабильный сервисный аккаунт, который работает годами с предсказуемым поведением. ИИ-агент находится между этими моделями: не человек, способный самостоятельно оценить допустимый объём своих полномочий, и не полностью статичный сервис с неизменным поведением. Как отмечает [исследование Cloud Security Alliance, проведённое при поддержке Aembit](#), существующие подходы к IAM испытывают нагрузку, на которую изначально не проектировались.

Три цифры важнее остальных. По февральской волне отчёта [Gravitee State of AI Agent Security 2026](#), лишь 21,9% команд рассматривают агента как самостоятельную сущность с собственной идентичностью. При этом 45,6% используют общие API-ключи для взаимодействия между агентами, а 44,4% — универсальные токены. По тому же исследованию CSA и Aembit, 68% организаций не могут чётко отличить действия ИИ-агентов от действий людей; там же 74% отмечают, что агенты нередко получают избыточный доступ, а 52% фиксируют наследование прав, изначально предназначенных людям или другим системам.

Для контекста, не для доказательства тезиса: по апрельской волне того же отчёта Gravitee в среднем мониторингом и средствами защиты охвачено около 52% развёрнутых агентов, а примерно 48% агентского парка остаётся вне такого покрытия — при этом средний агентский парк в исследованной выборке за четыре месяца почти удвоился, а лишь 9,5% организаций защищают больше 80% своего парка. По данным [Zscaler ThreatLabz 2026](#) — 989,3 млрд AI/ML-транзакций за 2025 год, рост 83% год к году в экосистеме из более чем 3400 приложений; это весь корпоративный AI/ML-трафик, включая встроенный ИИ в SaaS, а не только автономные агенты.

Сведём Zero Trust к трём рабочим принципам: явная верификация, минимальные привилегии, презумпция взлома. Упрощение — в самом NIST 800-207 положений больше и сформулированы они иначе, — но для практического разбора оно рабочее.

Явная верификация. Сложность агента не в том, что он работает долго: сервисные аккаунты тоже работают годами. Сложность в том, что один и тот же агент в рамках одной цепочки может последовательно выступать читателем данных, инициатором действия, делегатом полномочий и клиентом нескольких внешних инструментов. Каждый вызов инструмента

должен сопровождаться проверяемой идентичностью агента и отдельным решением об авторизации, с учётом ресурса, операции, задачи и инициатора. Новый интерактивный вход при каждом вызове для этого не нужен — но полномочия должны быть короткоживущими, узко ограниченными и проверяемыми в контексте конкретной операции, а их дальнейшую выдачу должно быть можно оперативно прекратить. Реализацией могут быть scored-токены, mTLS на основе короткоживущих X.509-SVID, JWT-SVID в модели SPIFFE/SPIRE или другие сопоставимые workload credentials.

Минимальные привилегии. Право должно быть привязано не к тому, что в принципе мог бы обосновать владелец агента, а к конкретной задаче, которую агент решает прямо сейчас. У каждого логического агента — собственная базовая машинная идентичность, а у каждого запуска или делегированной подзадачи — короткоживущий производный контекст, позволяющий восстановить цепочку: человек → агент → субагент → инструмент.

Презумпция взлома. Если поведением агента удалось манипулировать — например, через непрямую prompt injection, один из ключевых практических рисков агентных систем, относящийся к [LLM01:2025 Prompt Injection](#) в OWASP Top 10 for LLM Applications, — либо его runtime или учётные данные были скомпрометированы, ущерб должен быть технически ограничен архитектурой, а не только политикой на бумаге.

Три слоя контроля Zero Trust для ИИ-агентов

Идентичность → авторизация → точки применения политики

1

ИДЕНТИЧНОСТЬ

Workload identity, инициатор, instance/run ID, цепочка делегирования

2

АВТОРИЗАЦИЯ

Ресурс, операция, задача, TTL, policy decision, least privilege

3

POLICY ENFORCEMENT

NGFW / VLAN, NetworkPolicy, service mesh, egress, API- и MCP-шлюзы

Схема 1. Идентичность, авторизация и независимые точки применения политики.

Мы у себя в «Смарт-Софт» разбирали эту модель не как абстракцию, а как практическую архитектурную задачу. Схема получается трёхслойной. Первый слой — идентичность: каждый логический агент должен быть атрибутируемым машинным субъектом, общие ключи, не позволяющие различить исполнителей, следует исключать. Второй слой — авторизация на уровне инструмента: не «агент имеет доступ к CRM», а «агент имеет доступ к операции чтения одной конкретной таблицы CRM в рамках одной задачи». Третий слой — точки применения политики: в классической инфраструктуре это сеть и NGFW, в Kubernetes — network policy, service mesh и egress gateway, для внешних инструментов — API- или MCP-шлюз.

Разберём на конкретном сценарии. Коротко в терминах модели угроз: активы — репозиторий и тикет-система, доверенная граница — agent runtime, атакующий контролирует содержимое тикета, цель атакующего — заставить агента вызвать неразрешённый инструмент или вывести данные на внешний узел.

Агенту нужно прочитать тикет в трекере, найти связанный коммит в репозитории и оставить комментарий с диагнозом. Наивная реализация — токен с правами «читать/писать в трекере» и «читать репозиторий» на уровне всего проекта, трафик через общий исходящий шлюз вместе с остальным офисным интернетом. Если в тикете окажется спрятанная инструкция, текстом, неотличимым от обычного технического комментария, агент выполнит её с теми же правами, что и легитимную задачу, а трафик уйдёт туда же, куда уходит обычный офисный браузеринг.

Рабочая версия выглядит иначе. В журнале авторизации и делегирования одна операция могла бы быть представлена примерно так (часть полей — в claims токена, часть — в записи authorization server, часть — в distributed trace, не обязательно всё в одном месте):

```
actor_id:      agent-ticket-diagnoser
instance_id:   run-8f23
initiated_by:  user-1842
parent_agent:  orchestrator-17
task_id:       INC-4312
tool:          repository.read
resource:      repo/backend-service
credential_ttl: 300s
decision:      allow
```

Сетевой доступ агента к репозиторию идёт через выделенный сегмент со своей политикой на NGFW: разрешён только протокол и хост системы контроля версий. С оговоркой — это ограничивает направления связи, но не заменяет репозиторные ACL и авторизацию операций на уровне API. [Traffic Inspector Next Generation](#) в этой архитектуре отвечает за сетевой слой: применяет правила по интерфейсам и VLAN (IEEE 802.1Q), журналирует трафик, соответствующий настроенным правилам, может передавать сообщения журналов на внешний Syslog-сервер и формировать потоковую телеметрию NetFlow. Это два разных класса телеметрии, отвечающих на разные вопросы. Не реальный захват и не дословный формат журнала TING, а иллюстрация того, как после нормализации на Syslog-сервере или в SIEM могло бы выглядеть заблокированное обращение агента туда, куда не следовало:

```
src=10.4.12.7 dst=203.0.113.44 dst_port=443
proto=tcp interface=vlan-agents-42
action=block rule=agent-egress-default-deny
```

Syslog в этом примере отвечает на вопрос «какое правило сработало». NetFlow отвечает на другой вопрос — «как выглядел сетевой обмен»: метаданные разрешённых или фактически наблюдавшихся потоков, направления, объёмы, длительность и частота соединений. Само по себе заблокированное обращение не доказывает инъекцию: индикатор отклонения от ожидаемого поведения может объясняться и багом конфигурации, и галлюцинацией агента, и много чем ещё. Но это независимый источник телеметрии, который можно сопоставить с тем, что рассказывает сама агентская платформа, если её собственные логи неполны.

Зона ответственности NGFW заканчивается на сети. Идентичность агента и авторизация на уровне конкретного вызова инструмента — задача IAM и платформы, на которой работает агент, не сетевая. Zero Trust для агентов — это всегда стек из нескольких уровней контроля, а не одна коробка на периметре.

Вопрос, который мы для себя окончательно не закрыли: агент, порождающий субагента для подзадачи, должен ли передавать ему идентичность и права родителя, или субагент должен получать собственные? Полное наследование размывает атрибуцию действий — если субагент скомпрометирован, он действует от имени родителя, и в логах это может быть неотличимо от легитимного вызова. Полностью независимая идентичность честнее с точки зрения модели

угроз, но резко увеличивает операционную сложность при десятках динамически порождаемых субагентов за сессию. Мы склоняемся к третьему варианту: субагент получает собственный идентификатор и короткоживущие полномочия, а система сохраняет проверяемую связь с родителем, исходным пользователем и конкретной задачей — в claims, журнале делегирования или связанном контексте авторизации, без привязки к одному конкретному формату токена. Вопрос к тем, кто уже такое строил: где вы храните и проверяете саму цепочку делегирования — в IAM, на API-шлюзе, в agent runtime или на уровне MCP-шлюза?

68% компаний не могут чётко отличить действия агента от действий человека. Это не решается покупкой одного продукта, а начинается с управленческого решения: агент — отдельная идентичность, а не расширение прав того, кто его запустил. Вопрос к вам: в вашей инфраструктуре ИИ-агенты уже выданы как отдельные идентичности, или пока прячутся за чужими токенами, и вы узнаете об этом только при разборе инцидента?

Источники и материалы по теме

1. [NIST Special Publication 800-207, «Zero Trust Architecture»](#) — раздел 5.7 «Use of Non-person Entities (NPE) in ZTA Administration».
2. [Gravitee, «State of AI Agent Security 2026»](#) — волна февраль 2026: 21,9% identity-bearing entities, 45,6% shared API keys, 44,4% generic tokens; волна апрель 2026: 52% mean monitoring coverage, 48% unsecured, 9,5% securing >80%.
3. [Cloud Security Alliance / Aembit, «More Than Two-Thirds of Organizations Cannot Clearly Distinguish AI Agent from Human Actions»](#) — 68% не различают действия агента и человека, 74% отмечают избыточный доступ, 52% — наследование прав.
4. [Zscaler, «ThreatLabz 2026 AI Security Report»](#) — 989,3 млрд AI/ML-транзакций за 2025 год, рост 83% год к году, более 3400 приложений.
5. [OWASP, LLM01:2025 Prompt Injection](#) — OWASP Top 10 for LLM Applications.
6. [Traffic Inspector Next Generation, официальные характеристики](#) — VLAN (IEEE 802.1Q), stateful-фильтрация, правила по интерфейсам, журналирование, NetFlow, системный журнал и syslog-ng.

Примечание

Иллюстрации сетевых журналов в статье являются условными нормализованными примерами и не воспроизводят дословный формат интерфейса или журнала Traffic Inspector Next Generation.